



BSc Hons in Computing

2025.1

IOS Course Work Proposal

Coventry Index	NIBM Index	NIBM Registered Name
16602614	COBSCCOMP251P-015	A A D MALAN

Faculty of Engineering, Environment and Computing
School of Computing, Electronics and Mathematics
Coventry University
School of Computing and Engineering
National Institute of Business Management
Colombo-7

Contents

1. App Idea	3
Problem and value proposition.....	4
2. Feature Descriptions	4
1. Authentication & Onboarding.....	4
2. Home / Dashboard	4
3. Vehicle Listing & Search.....	4
4. Vehicle Details	4
5. Booking & Pricing.....	5
6. Driver License OCR	5
7. Face ID Security	5
8. Push Notifications	5
9. Smart Pickup & Return.....	5
10. AR 3D Vehicle Preview	5
11. Rental Status Widget	5
12. Driving Statistics	6
13. Profile, Settings, History & Payment.....	6
3. UI/UX Prototype	6
iOS design standards applied.....	7
4. Tools and Services Justification.....	7
5. Proposed Scope and Delivery Approach	8
Success criteria	8
6. LLM Use Statement	9
7. Conclusion	9

iOS Vehicle Rental Application Proposal

A high-fidelity, iOS-first rental experience combining booking, identity verification, smart pickup, AR preview and driving insights.

PLATFORM	iOS
PRODUCT TYPE	Vehicle rental mobile application
PROTOTYPE	Styled iOS Prototype (Figma)
PROPOSAL SCOPE	MVP + selected advanced iOS capabilities

PROPOSAL FOCUS

A practical MVP for searching, booking and managing rental vehicles, enhanced by Apple-native capabilities such as Face ID, Vision OCR, MapKit/Core Location, ARKit/RealityKit and WidgetKit.

1. App Idea

QuickRentCar is an iOS vehicle-rental application designed to make the rental journey faster and more self-service, from discovering a vehicle to completing pickup and monitoring an active rental. The target audience is customers who prefer to search, compare and book rental vehicles from their iPhone, including local users and visitors who need a simple mobile rental experience. The app combines vehicle discovery, pricing, booking, identity verification, pickup/return guidance, rental status and account management in one consistent interface.

The main unique selling point is the use of iOS-native features to reduce friction during the rental process. Driver-license details can be captured using Vision OCR, Face ID can secure account access, MapKit and Core Location can support smart pickup/return and geofencing, AR can preview a vehicle in the user's real environment, and WidgetKit can surface active-rental information without opening the app. A driving statistics view adds useful rental insight such as total distance, daily/weekly/monthly distance and speed analytics.

Problem and value proposition

User problem	QuickRentCar response	User value
Rental steps are fragmented across calls, forms and counters.	A single iOS flow for search, booking, verification, pickup and rental management.	Less manual coordination and a clearer rental journey.
License details are time-consuming to enter manually.	Vision OCR scans and extracts key license fields for confirmation.	Faster onboarding with fewer typing errors.
Customers may be unsure where to collect or return a vehicle.	MapKit, Core Location and geofence status show vehicle, pickup and return locations.	Clear navigation and location-aware guidance.
Active-rental information is easy to miss.	Push notifications and a WidgetKit rental-status widget surface key updates.	Timely information without repeatedly opening the app.

2. Feature Descriptions

The MVP follows the handwritten feature index supplied for the project: authentication/onboarding, settings/profile, dashboard, Firebase/API integration, push notifications, Face ID, booking, statistics, and history/payment. The advanced layer extends the experience with Apple Vision OCR, MapKit/Core Location, AR preview and WidgetKit.

1. Authentication & Onboarding

How it works: Users create or access an account and complete the minimum information required to use the rental service. The onboarding flow introduces the main rental journey and prepares the user for license verification and booking.

Problem solved: Creates a controlled starting point for customer identity and account-based bookings.

2. Home / Dashboard

How it works: The home screen provides quick access to pickup location, rental dates, vehicle search, categories, featured vehicles and shortcuts to advanced features.

Problem solved: Reduces the number of steps needed to begin a vehicle search or return to an important rental action.

3. Vehicle Listing & Search

How it works: Vehicles are presented with image, name, daily price, passenger capacity, fuel type, transmission/seats and included mileage. Category filters help users narrow results.

Problem solved: Makes comparison easier and gives users essential rental information before opening a detail page.

4. Vehicle Details

How it works: A selected vehicle has a larger visual presentation, specifications, price-per-day information, included mileage and access to booking or AR preview.

Problem solved: Provides enough detail for an informed rental selection without requiring external communication.

5. Booking & Pricing

How it works: Users choose pickup/drop-off dates and times, pickup location and review the calculated rental price before confirmation. The booking record is stored against the customer profile.

Problem solved: Turns vehicle discovery into a clear, traceable reservation flow and reduces ambiguity about price and rental period.

6. Driver License OCR

How it works: The camera captures a driver license. Apple Vision OCR extracts name, date of birth, license number and expiry date, then presents the values for user confirmation/editing before submission.

Problem solved: Reduces repetitive manual entry and helps standardize customer verification data.

7. Face ID Security

How it works: Face ID can be used for secure re-entry or protection of sensitive account actions through Apple LocalAuthentication. A fallback device passcode flow can be retained where appropriate.

Problem solved: Adds convenient device-level authentication without requiring the user to repeatedly type credentials.

8. Push Notifications

How it works: Rental reminders, booking status changes, pickup/return reminders and other time-sensitive updates are delivered through push notifications.

Problem solved: Keeps customers informed about actions that may otherwise be missed.

9. Smart Pickup & Return

How it works: MapKit displays pickup/return locations and vehicle/user position. Core Location supports distance calculation, navigation hand-off and geofence-based arrival/return-zone status. Nearby fuel or charging stations may also be shown.

Problem solved: Helps customers locate the vehicle and confirm they are within the correct pickup or return area.

10. AR 3D Vehicle Preview

How it works: Using ARKit/RealityKit, the user can place a 3D vehicle representation in a real environment, then rotate or resize it before continuing to booking.

Problem solved: Provides a more engaging way to understand vehicle scale and appearance before renting.

11. Rental Status Widget

How it works: WidgetKit surfaces active or upcoming rental information such as vehicle, status, return countdown and location summary from the iOS Home Screen.

Problem solved: Gives immediate access to important rental information without launching the full app.

12. Driving Statistics

How it works: A dedicated Stats tab presents a focused analytics view: one total-distance card, one daily/weekly/monthly distance chart and one speed analytics component showing average and maximum speed. It can also show used versus available daily kilometres when the rental includes a mileage allowance.

Problem solved: Helps customers understand rental usage and remaining allowance during the booking period.

13. Profile, Settings, History & Payment

How it works: The profile area stores customer details and settings, while booking history provides access to previous/current rentals and related payment records. A payment provider is not fixed in the supplied MVP notes and can be selected during implementation.

Problem solved: Centralizes account management and gives users a record of rental activity and charges.

3. UI/UX Prototype

The high-fidelity prototype is designed as an iOS-first interface with a clean blue-and-white visual system, rounded cards, clear hierarchy, large touch targets and a persistent five-item bottom navigation: Home, Bookings, AR, Stats and Profile. The prototype is intended to demonstrate the expected visual language, screen-to-screen flow and key interactions before implementation.

Figma prototype: [QuickRentCar – Styled iOS Prototype](#)

Figma prototype view : [Figma-Prototype-View](#)

Prototype area	Key interaction	Expected transition / feedback
Home	Select location/dates, search vehicles, open featured vehicle.	Push to listing/details; selected inputs remain visible.
Vehicle listing	Filter category, review vehicle cards, tap View Details or Book.	Card selection opens vehicle details with contextual back navigation.
Vehicle details	Review image/specifications, open AR, or continue to booking.	Smooth push transition; primary CTA remains visually dominant.
Booking	Choose rental period/location and review price calculation.	Validation before confirmation; success state after completed booking.
License OCR	Position license in camera frame, scan, review extracted fields.	Scanning feedback followed by editable extracted-details confirmation.
AR preview	Place, rotate and resize the vehicle in the camera view.	Direct manipulation gestures with a clear Book This Vehicle CTA.
Pickup / Return	View map, distance, route and geofence status.	Location updates change arrival/return-zone state and enable next action.
Stats	Switch Daily/Weekly/Monthly and review distance/speed.	Segment selection updates the single chart and summary values.

Prototype area	Key interaction	Expected transition / feedback
Profile	Manage account/settings and open history/payment records.	List-style iOS navigation into secondary account screens.
iOS design standards applied • Use familiar iOS navigation patterns, safe-area spacing and predictable back navigation. • Keep primary actions visually prominent and reachable with one-handed interaction where practical. • Use readable typography, sufficient contrast, concise labels and consistent iconography. • Provide visible system states for loading, scanning, success, errors, location permissions and unavailable services. • Use native permission prompts only when a feature needs them (camera, location, notifications, Face ID). • Prefer native gestures and transitions so the interface feels consistent with other iOS applications.		
4. Tools and Services Justification		
Tool / service	Planned use	Justification
Xcode	Primary IDE for iOS development, debugging, simulator/device testing, signing and distribution.	Apple's official development environment and required for building and testing native iOS apps.
Swift	Primary programming language.	Modern, type-safe and fully supported across Apple frameworks used by the application.
SwiftUI	Main UI framework for screens and reusable components.	Supports declarative iOS interfaces, rapid iteration and consistent state-driven UI.
Firebase	Backend services for authentication, cloud data storage and app-side integration/API needs.	Matches the MVP note and reduces the amount of custom backend infrastructure required for an academic MVP.
Firebase Cloud Messaging + APNs	Push-notification delivery.	FCM can coordinate notification delivery while APNs is the native Apple delivery channel for iOS devices.
LocalAuthentication	Face ID / device authentication.	Apple-native framework for secure biometric authentication without directly handling biometric data.
Vision / VisionKit	Driver-license OCR and camera-assisted document capture.	Provides on-device text recognition and Apple-native vision capabilities suitable for extracting license fields.
MapKit	Map display, annotations, routes and place information.	Native iOS mapping framework with strong integration into the platform.
Core Location	User/vehicle location, distance and geofence monitoring.	Supports location-aware pickup and return logic, including region monitoring.
ARKit + RealityKit	3D vehicle placement and manipulation in the real world.	Apple-native stack for reliable motion tracking, scene placement and 3D rendering on supported devices.
7		

Tool / service	Planned use	Justification
WidgetKit	Rental-status Home Screen widget.	Provides glanceable iOS widgets that can surface upcoming/active rental information.
Swift Charts	Distance and speed visualization in the Stats screen.	Native charting integrates cleanly with SwiftUI and is sufficient for the focused analytics requirement.
Figma	High-fidelity UI/UX design and interactive prototype.	Allows the app flow, visual system and interactions to be validated before development.

5. Proposed Scope and Delivery Approach

The project should prioritize a stable end-to-end rental flow before advanced capabilities. The MVP establishes the account, vehicle, booking, notification, security, statistics and history/payment foundation. Advanced features can then be integrated individually so each one can be demonstrated and tested without destabilizing the core booking experience.

MVP foundation	Advanced iOS enhancement
• Authentication & onboarding	• Vision OCR license scan
• Settings / profile	• MapKit + Core Location smart pickup/return
• Home dashboard	• ARKit/RealityKit 3D vehicle preview
• Firebase/API integration	• WidgetKit rental-status widget
• Push notifications	
• Face ID	
• Vehicle search/details	
• Booking & pricing	
• Stats	
• History & payment records	

Success criteria

- A user can discover a vehicle, review details, select rental dates/location, see pricing and complete a booking.
- License OCR can capture the required fields and lets the user confirm or correct them before saving.
- Pickup/return screens communicate location, distance and geofence status clearly.
- The Stats tab presents a simple, readable total-distance card, one distance chart and one speed analytics component.

- Active rental information can be surfaced through notifications and the rental-status widget.
- The prototype and implementation remain visually consistent with iOS interaction conventions.

6. LLM Use Statement

I used **ChatGPT** as a Large Language Model (LLM) to support non-code activities during the planning and design of the QuickRentCar iOS application. It was used for **brainstorming application features, refining the MVP scope, organizing feature descriptions, improving proposal wording, and generating ideas for the UI/UX structure and user flow.**

The LLM-generated suggestions were used as a **supporting reference rather than being accepted directly.** I reviewed, selected, and modified the outputs to match the application requirements, my own design decisions, and the intended iOS user experience. The final prototype structure, feature selection, and submitted content were reviewed and adapted by me.

7. Conclusion

QuickRentCar is positioned as a practical iOS rental application with a clear MVP and a set of platform-specific enhancements that demonstrate meaningful use of Apple technologies. The core product solves the everyday problems of vehicle discovery, booking, customer verification and rental management, while the advanced features improve convenience and differentiation without replacing the essential booking workflow. This structure keeps the project implementable while still providing strong technical depth for an iOS application proposal.

MVP

Index 25.1/015

- ① Auth / on board
- ② Settings / Profile
- ③ Dash board
- ④ API (fire base)
- ⑤ Push notification
- ⑥ Face ID
- ⑦ Booking. (Customer)
Prims
- ⑧ Statr.
- ⑨ Histograms Pers.

Advanced Features

- ① vision [exp], [6]
- ② map lat [4] - Live time. 2?
width lat [2]
- [4] - Geo fence?